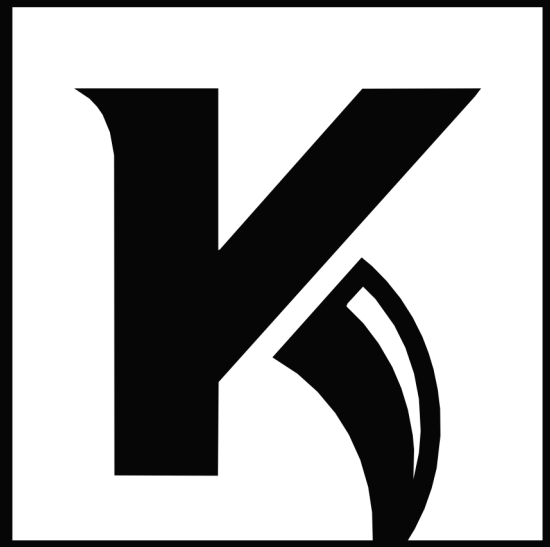
KUJO

Trust at the speed of AI.

The programming language and accountability infrastructure for agent-driven development.



Software production is outpacing verification.

Agents can report completion. Teams still need a reviewable chain from requested work to release decision.

01 **The claim**

“Tests passed. Ready to merge.” An illustrative agent statement.

02 **The evidence**

Command, exit code, changed revision, environment and retained failure output.

03 **The gap**

Useful logs already exist. Connecting them across tools and review boundaries remains work.

Greater autonomy requires stronger accountability.

The cost of an unverified action grows as agents gain authority.

01

Suggest code

Human selects the change → diff review

02

Complete a task

Agent edits and runs commands → execution records

03

Operate a workflow

Retries and external effects → boundaries, recovery and approval

AI use is broad. Trust remains conditional.

Adoption of AI assistance is not evidence of autonomous production deployment.

65.9%

use coding assistants or coding agents at work

26.2%

use AI agents or automated workflows at work

48%

trust AI when answers are easy to validate

Build with clarity. Operate with control.

The language is the long-term foundation. Existing tools offer an earlier route into engineering workflows.

01 **Clarity**

Kujo language / Spec / Eval: explicit intent and testable outcomes.

02 **Context**

MCP / CaseFile / RunLedger: useful inputs, debugging evidence and run records.

03 **Control**

Dispatch / Workcell / ShipCheck: orchestration, bounded execution and release checks.

Existing repositories + coding agents

explicit component contracts

Kujo runtime + CLI

Accountability spans the whole task.

Each component owns a specific responsibility. A reviewable workflow composes their artifacts explicitly.

01 **Instruction → Context**

Spec / PackWrite → MCP and repository context

→ 02 **Execution → Evaluation**

Dispatch / Workcell → Eval result artifacts

→ 03 **Evidence → Review → Release**

CaseFile / RunLedger → reviewer → ShipCheck gate

Human authority remains explicit at the release decision.

The foundation is already being built.

Inspect the implementation and its limits, then reproduce the smallest useful proof.

01 **Language and VM**

Kujo 1.9.0 is documented; a local 1.9.0 binary executes the recorded fixture tooling.

02 **Bounded execution**

Workcell 1.3 documents Docker/Podman limits, explicit exports and receipt verification.

03 **Reviewable artifact**

This presentation uses the native Kujo builder. Its demo retains a failing command, correction and evaluation.

A claim becomes inspectable evidence.

Recorded local coding fixture: correct a post-money ownership calculation. Inspect what failed, what changed and what passed.

01 **Request**

Return 20% for $6M raised at $30M post-money.

02 **Execution**

CaseFile captures real Node assertion failure and corrected execution.

03 **Decision**

Eval checks the correction. Accept this fixture; no production release.

Adopt the ecosystem before the language.

Keep application code in its existing language. Introduce accountability at the workflow boundary.

01

Today: wrap the task

Use CaseFile for command evidence and Eval for explicit repository checks.

02

Then: control execution

Adopt Dispatch or Workcell after validating the target environment and runtime pins.

03

Over time: go deeper

Choose Kujo for new agent tooling where native language integration adds value.

Start with teams already delegating engineering work.

Proposed beachhead: agent-platform builders shipping repeatable engineering workflows.

01 **Buyer / user**

Platform engineering lead / engineers operating coding-agent harnesses.

02 **Immediate pain**

Failed runs are expensive to reconstruct. Review spans commits, logs and agent transcripts.

03 **Purchase hypothesis**

Pay for retained evidence, controlled execution and team review if they reduce investigation time.

Open tooling can lead to a commercial operating layer.

Earn developer trust through useful local tools. Charge for shared operational responsibilities.

01

Available foundation

Open repositories and local developer tooling.

02

Proposed team service

Hosted evidence, retention, shared review and execution capacity.

03

Proposed enterprise tier

Organization policy, deployment controls, support and governance.

A focused entry market can support a meaningful business.

Illustrative bottom-up model. Organization counts and annual contracts are assumptions to validate.

2,000

initial target organizations × $24K ACV

$48M

illustrative annual serviceable revenue pool

$2.4M

100 customers × $24K; milestone scenario, not forecast

Accountability requires more than generating code.

Existing platforms are strong. Kujo must win on useful evidence and the cost of operating reliable workflows.

01 **Coding agents / GitHub**

Generation, repository context and review are already integrated into major developer surfaces.

02 **Orchestration / LangGraph**

Persistence, recovery and human review are established capabilities.

03 **Kujo opportunity**

Connect explicit contracts, bounded work and inspectable artifacts. Prove adoption before claiming a moat.

Meet developers inside their existing workflows.

The planned December 2026 open-source campaign should produce repeat usage and customer learning.

01

Discovery

GitHub releases, technical demos, docs and reproducible fixtures → qualified visits.

02

Activation

Agent harness recipes and MCP tooling → first successful evidence run.

03

Retention → revenue

Repeat weekly runs → design-partner discussions → paid pilots and renewal evidence.

Execution is visible. Demand still needs proof.

Engineering output, external adoption and commercial traction are different measures.

01 **Engineering output**

Public repositories, documented releases and the locally reproduced evidence fixture.

02 **External adoption**

Verified outside usage, retention and independent integrations: data not supplied.

03 **Commercial traction**

Design partners, paying customers and revenue: data not supplied. Complete before investor distribution.

Built through sustained open-source engineering.

Robert DeVore connects open-web product work with a growing language and developer-tooling ecosystem.

01 **Background**

Public biography describes years of WordPress, WooCommerce, plugins and publishing systems.

02 **Current work**

Language/runtime infrastructure, orchestration, evaluation, telemetry and evidence workflows.

03 **Team enabled by capital**

Proposed hires in runtime, execution reliability and developer experience; later customer engineering.

Capital turns a technical foundation into an adopted platform.

Working seed scenario. Financing terms and hiring pace remain assumptions.

$6M

target raise

$30M

post-money valuation or SAFE cap assumption

24 mo

planned runway, including reserve

0–6 mo
Flagship evidence workflow

6–12 mo
External validation + pilots

12–24 mo
Paid retention + repeatability

Agents will build more software.

The organizations deploying it will still be accountable for what it does.

KUJO

Trust at the speed of AI.

Investor conversations ↗